API DEI GIOCHI BM, VOLUME 4

Le `librerie` bmlib e bmnet

La libreria condivisa per ciò che ogni gioco riscrive da capo, e la libreria per i giochi in rete.

Indice

Le due librerie 4

Cosa sono 4

Come caricarle 4

Da sapere su bmlib 4

Da sapere su bmnet 4

Tutorial: un passo di platform con bmlib 6

Prima di iniziare 6

Passi 6

Tutorial: un gioco a due in rete con bmnet 8

Passi 8

bmlib: numeri 10

bmlib: numeri casuali 11

bmlib: collisioni 12

bmlib: la mappa, muri, piattaforme, gravità 13

bmlib: hitbox e hurtbox 14

bmlib: easing, tween, timer e script 16

bmlib: liste e particelle 17

bmlib: la camera 18

bmlib: stati di gioco 19

bmlib: testo, barre e menu 20

bmlib: giocatori su una console 22

bmlib: suono, salvataggi, animazioni, colori 24

bmlib: il costruttore 3D 25

bmnet: aprire la rete e la lobby 26

bmnet: messaggi ed eventi 27

bmnet: la partita in lockstep 28

bmnet: campi ed esempio completo 29

Campi 29

Un esempio completo 29

Provarlo sul PC 30

Punti aperti 31

LEGGIMI

Cos'è. Il volume 4 dell'API dei giochi bm. Descrive le due librerie standard del kernel, `bmlib` e `bmnet`: ogni funzione, con la sua firma e ciò che fa.

Come usarlo. Leggi una volta i primi tre capitoli: caricano le librerie e costruiscono, passo per passo, un piccolo platform e un gioco a due in rete. Poi usa gli altri capitoli come riferimento, un'area alla volta.

Cosa manca. Ogni lacuna è segnata con un'etichetta evidenziata DA COMPLETARE; nulla è inventato.

Le due librerie

Cosa sono

`bmlib` è ciò che ogni gioco scriveva da sé, ora in una sola libreria scritta in Lua e tenuta nel kernel (`src/script/bmlib.lua`). Comprende: numeri e numeri casuali, collisioni, la mappa (muri, piattaforme, gravità), hitbox e hurtbox, easing, tween, timer e script, particelle, una camera, stati di gioco, testo, menu e menu di pausa, giocatori su una console, jingle, salvataggi, animazioni, colori e un costruttore di mesh 3D.

`bmnet` è il codice di rete del gioco Overbit, messo a disposizione di ogni gioco (`src/script/bmnet.lua`). Le console si trovano nella **LAN** (broadcast UDP) o su **internet** attraverso un relay; una console **ospita** una partita e le altre **entrano**; si scambiano **messaggi** e, per i giochi d'azione, la partita gira in **lockstep**.

Come caricarle

```
local lib = require "bmlib"
local net = require "bmnet"
```

`bmlib` non tocca le variabili globali e non nasconde nessuna funzione della console.

Da sapere su bmlib

- I tempi sono in **secondi**, le posizioni in pixel, le velocità in **pixel per frame**.
- Gli oggetti (particelle, camera, stati, menu, pausa, costruttore 3D) si usano con i due punti: `cam:follow(x, y)`.
- Tween, timer, script e jingle vanno avanti con `lib.update()`, da chiamare una volta in `_update`. Aggiunge 1/60 s; `lib.update(dt)` aggiunge un altro passo. `lib.time` è il tempo contato così.

```
local lib = require "bmlib"

function _update()
  lib.update()          -- tween, timer, script e jingle
  -- ... il gioco
end
```

Da sapere su bmnet

- Chiama `net.update()` una volta in ogni `_update`: restituisce gli eventi del frame.
- Chiama `net.close()` quando il gioco finisce, e anche da `_leave()`.
- I pacchetti sono descritti in cima a `src/script/bmnet.lua`.

- Il template *Online 2D* dell'SDK è un esempio completo di gioco in rete. Il template *Versus 2D* è un esempio completo di giocatori su una console.

Tutorial: un passo di platform con bmlib

Costruisci un eroe che cammina, salta e atterra sulle tessere, con una camera che lo segue. Usi solo funzioni descritte in questo manuale.

Prima di iniziare

La mappa è disegnata sul livello 1 e le sue tessere portano dei **flag**. bmlib li legge così:

Flag	Significato	Costante
0	solido	`lib.SOLID`
1	piattaforma	`lib.PLATFORM`
2	scala	`lib.LADDER`
3	acqua	`lib.WATER`
4	fa male	`lib.HURT`

[DA COMPLETARE: come dipingere le tessere e impostare i loro flag negli editor; questo volume descrive solo la libreria]

Passi

1. Inizia un nuovo file Lua e carica la libreria: `local lib = require "bmlib"` .
2. Crea una tabella per l'eroe: `local hero = { x = 24, y = 24, w = 8, h = 8 }` . Un corpo è una tabella con `x, y, w, h` .
3. In `_init` , chiama `lib.tiles({ edge = true })` . Livello 1, flag solido 0 e flag piattaforma 1 sono i valori predefiniti; `edge = true` rende i bordi della mappa dei muri.
4. In `_init` , crea la camera: `cam = lib.camera({ bounds = true })` . Con `bounds = true` resta dentro la mappa.
5. In `_update` , chiama prima `lib.update()` .
6. Imposta la velocità dell'eroe dal pad: `hero.vx` vale 2 con `btn("right")` , -2 con `btn("left")` .
7. Salta solo quando `hero.ground` è vero: `hero.vy = -5` quando premi `btnp("a")` .
8. Imposta `hero.drop = btn("down")` , così l'eroe può scendere attraverso una piattaforma.
9. Chiama `lib.step(hero)` . Aggiunge la gravità, muove l'eroe, lo ferma contro le tessere solide e imposta `hero.ground` .
10. Chiama `cam:follow(hero.x, hero.y)` .
11. In `_draw` : `cls(0)` , `cam:apply()` , `cam:map()` , disegna l'eroe, poi `camera()` per il testo fisso.
12. Avvia il gioco. Dovresti vedere la mappa, l'eroe che cade sul terreno e lo schermo che lo segue quando cammini.

```
local lib = require "bmlib"
local hero = { x = 24, y = 24, w = 8, h = 8, vx = 0, vy = 0 }
local cam

function _init()
  lib.tiles({ edge = true })              -- i bordi della mappa sono muri
  cam = lib.camera({ bounds = true })     -- resta dentro la mappa
end

function _update()
  lib.update()
  hero.vx = (btn("right") and 2 or 0) - (btn("left") and 2 or 0)
  if hero.ground and btnp("a") then hero.vy = -5 end
  hero.drop = btn("down")                 -- giù da una piattaforma
  lib.step(hero)
  cam:follow(hero.x, hero.y)
end

function _draw()
  cls(0)
  cam:apply()
  cam:map()
  print("@", hero.x, hero.y, 0xFFFFFF)    -- l'eroe
  camera()
  print("A: jump", 4, 4, 0xFFFFFF)
end
```

[DA COMPLETARE: shots/api-lib-platformer.png]

PROVA

Cambia `hero.gravity` in 0.15, poi imposta `hero.maxfall` a 3 e salta di nuovo. L'eroe cade più lento, e mai oltre 3 pixel per frame.

Se l'eroe attraversa il pavimento, controlla che le tessere del pavimento abbiano il flag 0 (solido) e che siano sul livello 1, oppure passa il `layer` giusto a `lib.tiles` .

Tutorial: un gioco a due in rete con bmnet

Costruisci una lobby: una console ospita, l'altra entra, si parlano con i messaggi e poi avviano una partita. Usi solo funzioni descritte in questo manuale. Provalo su due console nella stessa LAN, oppure sul PC (vedi l'ultimo capitolo).

Passi

1. Carica le librerie: `local net = require "bmnet"` e `local lib = require "bmlib"`.
2. In `_init`, chiama `net.open({ game = "MYG1", name = "Ann" })`. `game` è di 4 caratteri: i pacchetti degli altri giochi non si vedono. Restituisce `true`, oppure `false` e il perché. Poi `net.state` vale `"lobby"`.
3. Aggiungi `function _leave() net.close() end`, così gli altri sanno quando te ne vai.
4. In `_update`, scorri `net.update()`: restituisce gli eventi del frame. Fallo una volta in ogni `_update`.
5. Sulla prima console, quando `net.state` è `"lobby"` e premi `btnp("a")`, chiama `net.host({ max = 2, info = "my match" })`. Questa console è il posto 1 e `net.state` diventa `"hosting"`.
6. Sulla seconda console, leggi `net.hosts()`: elenca le partite trovate negli ultimi 3 secondi. Mostrale e chiama `net.join(id)` per quella scelta.
7. Aspetta l'evento `"joined"` (è `"refused"` se la partita è piena o già iniziata). L'host riceve un evento `"join"` con il nome di chi è arrivato.
8. Di' qualcosa: `net.post("hello")` manda un messaggio **sicuro**. L'altra console riceve un evento `"msg"` con `from`, `data` e `sure`.
9. Sull'host, quando premi `btnp("start")`, chiama `net.start()`. Tutti ricevono l'evento `"start"` con `seed`, `seat` e `seats`.
10. Su `"start"`, crea il generatore casuale con `lib.rng(e.seed)`. Da ora `net.state` è `"playing"`.
11. In ogni `_update` della partita: `net.input(net.pad(1))`, poi `for f, inputs in net.frames() do ... end`. Muovi ogni giocatore da `inputs[seat]`, con `net.held(inputs[seat], "right")`; salta un posto il cui input è `false` (il giocatore se n'è andato).
12. Avvia su entrambe le console. Dovresti vedere la lobby, il nome dell'altro giocatore, i messaggi e i due quadrati che si muovono allo stesso modo su entrambi gli schermi.

```
local net = require "bmnet"
local lib = require "bmlib"
local rng
local chat = {}
local pos = { 40, 120 }                       -- una posizione per posto

function _init() net.open({ game = "MYG1", name = "Ann" }) end
function _leave() net.close() end

local function step(inputs)                   -- uguale su ogni console
  for seat = 1, 2 do
    local v = inputs[seat]
    if v then
      if net.held(v, "right") then pos[seat] = pos[seat] + 1 end
      if net.held(v, "left") then pos[seat] = pos[seat] - 1 end
    end
  end
end

function _update()
  for _, e in ipairs(net.update()) do
    if e.type == "join" then chat[#chat + 1] = e.name .. " joined"
    elseif e.type == "msg" then chat[#chat + 1] = e.data
    elseif e.type == "start" then rng = lib.rng(e.seed)
    elseif e.type == "lost" then chat[#chat + 1] = "host lost" end
  end
  if net.state == "lobby" then
    if btnp("a") then net.host({ max = 2, info = "my match" }) end
    for _, h in ipairs(net.hosts()) do
      if btnp("b") then net.join(h.id) end
    end
  end
  if net.state == "hosting" and btnp("start") then net.start() end
  if net.state ~= "off" and btnp("x") then net.post("hello") end
  if net.state == "playing" then
    net.input(net.pad(1))
    for _, inputs in net.frames() do step(inputs) end
  end
end
```

[DA COMPLETARE: shots/api-lib-lobby.png]

ATTENZIONE

In una partita il gioco deve dipendere solo dagli input dei frame e dal seed. Leggi le regole del lockstep nel capitolo "bmnet: la partita in lockstep".

PROVA

Avvia il gioco su due console. Premi A sulla prima, B sulla seconda, poi X su entrambe per vedere i messaggi, poi Start sulla prima.

bmlib: numeri

Funzione	Descrizione
`lib.clamp(v, lo, hi)`	`v` tra `lo` e `hi`
`lib.lerp(a, b, t)` / `lib.unlerp(a, b, v)`	da `a` (t = 0) a `b` (t = 1); l'inverso: dove si trova `v` tra `a` e `b`
`lib.remap(v, a0, a1, b0, b1)`	`v` dall'intervallo `a0..a1` all'intervallo `b0..b1`
`lib.approach(v, target, step)`	`v` verso `target` di al più `step` (barre che scendono piano, velocità che frenano)
`lib.sign(v)`	-1, 0 o 1 (0 per 0)
`lib.round(v, [step])`	all'intero più vicino, o al multiplo di `step` più vicino
`lib.wrap(v, lo, hi)`	`v` riportato dentro `lo..hi` (`hi` escluso) facendo il giro: mondi che si richiudono
`lib.cycle(i, d, n)`	l'indice `i` (1..n) spostato di `d` facendo il giro: le righe di un menu
`lib.dist(ax, ay, bx, by)` / `lib.dist2(...)`	la distanza / il suo quadrato (per confrontarlo con `r * r` senza la radice)
`lib.len(x, y)` / `lib.norm(x, y)`	la lunghezza di un vettore / il vettore lungo 1 (e la lunghezza che aveva); `0, 0, 0` per `0, 0`
`lib.angle(ax, ay, bx, by)`	l'angolo da `a` a `b` in radianti (0 a destra, π/2 in basso sullo schermo)
`lib.angdiff(a, b)` / `lib.turn(a, target, step)`	da `a` a `b` per la via più corta (-π..π) / l'angolo `a` ruotato verso `target` di al più `step`
`lib.dir8(x, y)`	una di 8 direzioni: 0 destra, 1 basso-destra, 2 basso... 7 alto-destra (`nil` per `0, 0`); `lib.DIR8[d + 1]` è il vettore lungo 1
`lib.TAU`	2π

bmlib: numeri casuali

Funzione	Descrizione
`lib.rnd([a, b])`	un numero tra `a` e `b` (`b` escluso); `lib.rnd(n)`: tra 0 e `n`; `lib.rnd()`: tra 0 e 1
`lib.chance(p)`	`true` con probabilità `p`
`lib.choose(t)` / `lib.shuffle(t)`	un elemento a caso della lista (`nil` se è vuota) / la lista mescolata (sul posto)
`lib.rng(seed)`	un generatore tutto suo, che dallo stesso seed dà sempre gli stessi numeri (mondi fatti da un seed, partite in rete lockstep): `r:next()` tra 0 e 1, `r:range(a, b)`, `r:int(a, b)` (interi, entrambi inclusi), `r:pick(t)`, `r:chance(p)`, `r:seed(s)`

bmlib: collisioni

Funzione	Descrizione
`lib.overlap(ax, ay, aw, ah, bx, by, bw, bh)`	due rettangoli (x, y, larghezza, altezza) si toccano
`lib.hit(a, b)`	lo stesso per due tabelle con `x, y, w, h` (`w` e `h` valgono 8 se mancano)
`lib.inside(px, py, x, y, w, h)`	il punto è nel rettangolo
`lib.circles(ax, ay, ar, bx, by, br)` / `lib.circrect(cx, cy, r, x, y, w, h)`	due cerchi / un cerchio e un rettangolo si toccano

bmlib: la mappa, muri, piattaforme, gravità

Le funzioni guardano i **flag delle tessere**: `lib.SOLID` (flag 0, valore 1), `lib.PLATFORM` (flag 1, 2), `lib.LADDER` (4), `lib.WATER` (8), `lib.HURT` (16). I flag scala, acqua e fa male servono al tuo codice: per esempio, `lib.ray(x0, y0, x1, y1, lib.HURT)` trova la prima tessera che fa male.

Funzione	Descrizione
`lib.tiles(opt)`	come la mappa ferma i corpi: `layer` (il livello: numero o nome, 1), `solid` e `platform` (le maschere dei flag: 1 e 2; 0 = nessuno), `edge` (`true` : fuori dalla mappa è solido). Restituisce la configurazione
`lib.solid(x, y, [w, h])`	il rettangolo (o il punto) tocca qualcosa di solido
`lib.move(b, dx, dy)`	muove il corpo `b = {x, y, w, h}` (`w` , `h` valgono 8 se mancano) di `dx, dy` , fermandolo contro le tessere solide e facendolo scivolare lungo i muri; cade su una piattaforma solo dall'alto (e non con `b.drop` : giù attraverso di essa). A passi più corti di una tessera: non ne salta mai una. Restituisce `hx, hy` : -1 / 1 dove ha urtato (sinistra / destra, su / giù), altrimenti 0
`lib.step(b)`	un frame di un corpo con gravità (un platform): `b.vx, b.vy` in pixel per frame, `b.gravity` (`lib.GRAVITY` , 0.25) e `b.maxfall` (`lib.MAXFALL` , 6); poi `b.ground` è `true` a terra e la velocità che ha urtato è 0. Restituisce ciò che dà `lib.move`
`lib.ray(x0, y0, x1, y1, [mask])`	la prima tessera con un flag di `mask` (quello solido di `lib.tiles` se manca) sul segmento: `x, y` dove il segmento vi entra e la sua cella `mx, my` ; `nil` se la via è libera (linea di vista, proiettili). Con `lib.tiles{edge=true}` anche ogni punto fuori dalla mappa conta come colpito, qualunque sia la maschera

```
local hero = { x = 40, y = 40, w = 8, h = 8, vx = 0, vy = 0 }
lib.tiles({ edge = true })                        -- i bordi della mappa sono muri
function _update()
  hero.vx = (btn("right") and 2 or 0) - (btn("left") and 2 or 0)
  if hero.ground and btnp("a") then hero.vy = -5 end
  hero.drop = btn("down")                         -- giù da una piattaforma
  lib.step(hero)
end
```

bmlib: hitbox e hurtbox

Le **hurtbox** sono dove un corpo può essere colpito, le **hitbox** dove un attacco fa male (un pugno, una spada, un proiettile). Un mondo di colpi le raccoglie in ogni frame e dice chi colpisce chi: squadre, un colpo per attacco, le parti del corpo, le corsie di un picchiaduro, lame che si scontrano. Le box sono date nel codice o vengono dallo sheet (`zboxes`).

Funzione	Descrizione
`lib.hits()`	un mondo di colpi `H` (tabelle `H.hurts` , `H.hitl` : le box del frame)
`H:clear()`	un nuovo frame, da chiamare in `_update` prima delle box: quelle di prima se ne vanno; gli attacchi (`id`) che non sono tornati nell'ultimo frame sono finiti
`H:hurt(who, x, y, w, h, [opt])`	una hurtbox di `who` (qualsiasi valore: la tabella del corpo). `team` (la stessa squadra non si colpisce), `part` (un nome: `"head"` ; prima le parti che contano di più), `z` e `depth` (una terza dimensione: la corsia, l'altezza)
`H:hit(who, x, y, w, h, [opt])`	una hitbox dell'attaccante `who` : `team` , `id` (un attacco: colpisce ogni corpo **una volta** finché lo stesso `id` torna frame dopo frame; le box di un attacco lo condividono; senza `id` fa male in ogni frame in cui tocca), `clash` (due hitbox con `clash` che si toccano fanno un contatto `"clash"`), `z` , `depth` e quel che serve al gioco (`damage` , `knock` ...)
`H:zone(who, name, frame, x, y, [flip, opt])`	le box del frame di una zona (`zboxes` : `hurt` e `hit`) per lo sprite disegnato con `zspr(name, x, y, frame, flip)` : le hurtbox con `opt` (`team` , `part`), le hitbox con `opt.attack` (le opzioni di `H:hit` ; la squadra di `opt` se manca)
`H:check()`	i contatti del frame, nell'ordine delle hitbox: `{kind = "hit" or "clash", by (l'attaccante), to (il corpo colpito, o l'altro attaccante), hit (le opzioni di H:hit), part, x, y (il punto medio di dove si toccano)}` . Un attacco colpisce un corpo una volta per frame (la sua prima box che tocca)
`H:draw([hurt_c, hit_c])`	le box, per vederle mentre fai il gioco (blu e rosso)
`lib.box(b, x, y, [flip, w])`	una box di un frame (`b.x` , `b.y` dall'angolo, `b.w` , `b.h`) nel mondo, per il frame disegnato in `x, y` ; con `flip` specchiata sulla larghezza `w` del frame: `x, y, w, h`
`lib.separate(a, b)`	due corpi `{x, y, w, h}` che si sovrappongono si allontanano lungo l'asse in cui si sovrappongono meno, metà ciascuno (`a.fixed` o `b.fixed` : solo l'altro); `true` se si toccavano. Per combattenti che non si attraversano, folle

```
local H = lib.hits()

function _update()
  H:clear()
  for _, f in ipairs(fighters) do
    H:zone(f, f.anim, f.frame, f.x, f.y, f.face < 0,
           { team = f.team, attack = { id = f.swing, damage = 5 } })
  end
  for _, c in ipairs(H:check()) do
    c.to.life = c.to.life - c.hit.damage    -- c.by ha colpito c.to
  end
end
```

bmlib: easing, tween, timer e script

`lib.ease` ha le curve da 0 a 1: `linear`, `inquad`, `outquad`, `inoutquad`, `incubic`, `outcubic`, `inoutcubic`, `insine`, `outsine`, `inoutsine`, `inback`, `outback`, `inoutback`, `outelastic`, `outbounce`, `smooth` (smoothstep).

Funzione	Descrizione
`lib.tween(obj, to, secs, [ease, done])`	i campi di `obj` vanno ai valori della tabella `to` in `secs` secondi (`ease`: una funzione o il nome di una di `lib.ease`, lineare se manca), poi `done(obj)`. Restituisce un handle: `h:cancel()`
`lib.after(secs, f)`	`f()` tra `secs` secondi
`lib.every(secs, f, [times])`	`f()` ogni `secs` secondi (`times` volte, per sempre se manca); se `f` restituisce `false` si ferma
`lib.script(f, ...)`	`f(...)` come **script che sa aspettare**: `lib.wait(secs)` (un frame se manca) e `lib.waitfor(cond)` (finché `cond()` è vera). Parte subito fino alla sua prima attesa, poi `lib.update()` lo porta avanti: dialoghi, cutscene, ondate di nemici scritte in fila
`lib.cancel(h)`	ferma un tween, un timer o uno script (come `h:cancel()`)
`lib.countdown(t, keys, [d])`	i campi `keys` di `t` sopra 0 scendono di `d` (1/60 se manca), non sotto 0: i tempi di ricarica di un eroe (in frame con `d = 1`)
`lib.clear()`	ferma ogni tween, timer, script e jingle (un nuovo livello)

Vanno avanti tutti solo con `lib.update()`.

```
lib.tween(title, { y = 80 }, 0.6, "outback")
lib.after(2, function() door.open = true end)
lib.script(function()
  say("Who goes there?")
  lib.wait(1.5)
  lib.waitfor(function() return btnp("a") end)
  say("You may pass.")
end)
```

bmlib: liste e particelle

Funzione	Descrizione
`lib.each(list, f)`	`f(item, i)` per ogni elemento; quelli per cui restituisce `false` escono dalla lista (gli altri mantengono l'ordine)
`lib.sweep(list)`	toglie gli elementi con `.dead` (in ordine)
`lib.particles([max])`	un pool di al più `max` particelle (200); quando è pieno una nuova prende il posto della più vecchia
`P:add(x, y, vx, vy, life, colour, [opt])`	una particella (velocità in pixel per frame, vita in secondi); `opt`: `size` (raggio; 0 o 1 un pixel), `gravity`, `drag` (la parte di velocità mantenuta a ogni frame), `colors` (una lista: il colore lungo la sua vita), `shrink`, `floor` (la y in cui rimbalza), `bounce` (0.3)
`P:burst(x, y, n, [opt])`	`n` particelle da `x, y`: `speed` (2), `angle` e `spread` (in radianti; tutto intorno se mancano), `life` (0.5 s), `color` o `colors`, e le opzioni di `:add`
`P:update()` / `P:draw()`	in `_update` / in `_draw`
`P:count()` / `P:clear()`	quante ce ne sono / via tutte
`lib.particles3d([max])`	la sorella 3D di `lib.particles`: un pool di al più `max` particelle (1024). Avvolge le funzioni delle particelle 3D (`particles`, `particles_add`, `particles_step`, `particles_draw`, `particles_count`)
`P3:add(x, y, z, vx, vy, vz, life, color)`	una particella 3D
`P3:burst(x, y, z, n, [opt])`	`n` particelle da `x, y, z`: `speed` (2), `life` (30 update), `color` (0xFFFFFF), `up` (0)
`P3:update([gravity])` / `P3:draw([radius])`	in `_update` (gravità 0.02) / in `_draw` (raggio 0.1)
`P3:count()`	quante particelle 3D sono vive

```
local fx = lib.particles(300)
fx:burst(x, y, 20, { speed = 3, colors = { 0xFFFFFF, 0xFFD050, 0xFF6020 }, gravity = 0.1 })

local p3 = lib.particles3d(500)
p3:burst(x, y, z, 20, { speed = 2, life = 30, color = 0xFFA020 })
p3:update()
p3:draw()
```

bmlib: la camera

Funzione	Descrizione
`lib.camera([opt])`	una camera 2D: `smooth` (0.15: la parte di strada che fa a ogni frame; 1 = si incolla al bersaglio), `dead` `{w, h}` (un riquadro al centro dove il bersaglio si muove senza la camera), `bounds` `{x0, y0, x1, y1}` in pixel o `true` (la mappa, `msize()`), `offset` `{x, y}`, `w`, `h` (lo schermo se mancano). Campi `x`, `y`
`C:follow(x, y, [snap])`	un passo verso il punto `x, y` (al centro dello schermo); `snap`: arriva subito
`C:shake(amount, [secs])`	lo schermo trema fino a `amount` pixel, sempre meno, per `secs` secondi (0.3)
`C:apply([view])`	`camera()` al posto della camera (con il tremolio): in `_draw` prima del mondo, poi `camera()` per l'HUD. Con una vista `{x, y, w, h}` (`lib.split`) il disegno resta nella vista (`clip`) e l'angolo della camera è quello della vista: dopo le viste, `clip()` e `camera()`
`C:map([layer, mask])`	le celle della mappa che si vedono
`C:sees(x, y, [w, h])` / `C:screen(x, y, [view])`	il rettangolo si vede / dove si trova sullo schermo (nella vista) un punto del mondo

bmlib: stati di gioco

`lib.states(defs, [first, ...])` : titolo, partita, pausa, game over come tabelle `{enter, update, draw, exit}` ; ogni funzione riceve la sua tabella (`function play:update()`), che ha `t` (i secondi passati nello stato) e `name` .

Funzione	Descrizione
`S:go(name, ...)`	allo stato `name` : quelli aperti escono (`exit`), `name` entra (`enter(...)`)
`S:push(name, ...)` / `S:pop()`	`name` sopra quello attuale (una pausa, un dialogo: aggiorna solo lui, disegnano entrambi) / indietro a quello sotto
`S:update()` / `S:draw()`	in `_update` / in `_draw` (dal fondo: il gioco sotto la sua pausa)
`S:is(name)` , `S.name` , `S:top()`	lo stato in cima

```
local S
S = lib.states({
  title = { update = function(s) if btnp("ok") then S:go("play") end end,
            draw = function() cls(0); lib.printc("PRESS A", 160, 0xFFFFFF) end },
  play = { enter = function(s) s.score = 0 end,
           update = function(s) s.score = s.score + 1 end,
           draw = function(s) cls(0) end },
}, "title")
function _update() lib.update(); S:update() end
function _draw() S:draw() end
```

bmlib: testo, barre e menu

Funzione	Descrizione
`lib.input([over])`	lo standard di input dei giochi: mette `confirm`, `cancel`, `pause` e le azioni dello schema del gioco nella keymap e restituisce la tabella. `over.scheme` sceglie lo schema: `"scroller"` (il predefinito: `act1` salto = B, `act2` attacco = Y, `act3` = il tasto A/B che non è ok, `act4` = X, spalle L1/R1), `"topdown"` (`act1` = B, `act2` = l'altro tasto A/B, `act3` = Y, `act4` = X) o `"stick"` (`act1` R2, `act2` L2, `act3` L1, `act4` R1, da `act5` a `act8` = B, A, Y, X). `lib.input_scheme` dice quale gira. Le altre chiavi di `over` (`{ act3 = "x" }`) cambiano singole azioni e sono elencate in `lib.input_exceptions`
`lib.textw(text, [scale])`	larghezza e altezza in pixel con il font di `print` in questo momento (la riga più lunga)
`lib.printc(text, y, [c, scale, x, w, grid])`	al centro dello schermo (o di `x..x + w`); `grid`: sulle colonne del font (testo che sta fermo, come i menu). Restituisce la x
`lib.printr(text, x, y, [c, scale])`	che finisce in `x` (numeri allineati a destra)
`lib.prints(text, x, y, c, [shadow, scale])` / `lib.printo(...)`	con un'ombra sotto a destra / con un contorno tutto intorno (nero se manca il colore)
`lib.bar(x, y, w, h, v, max, [c, back, border])`	una barra piena per `v` di `max` (vita, resistenza, caricamento)
`lib.blink([period, t])`	`true` per metà del tempo, cambiando ogni `period` secondi (0.5): "premi A"
`lib.timestr(secs, [tenths])`	`"m:ss"` (`"h:mm:ss"` da un'ora), con i decimi `"m:ss.d"`
`lib.btnr(i, [p, delay, rate])`	`btnp` che si **ripete** finché il tasto è premuto: il primo frame, poi dopo `delay` frame (15) ogni `rate` (4). `i` come per `btn()`. Chiamala in ogni frame
`lib.menu(items, [opt])`	una lista da cui scegliere: le voci sono testi o tabelle `{label=, value=, change=function(item, d), ok=function(item), off=true}`; `opt.p` il giocatore, `opt.wrap` (`true`)
`M:update()`	su e giù (con ripetizione) spostano, saltando le voci `off`, sinistra e destra chiamano `change`, ok sceglie (`ok(item)` ; restituisce la voce e `"ok"`), indietro restituisce `nil, "back"`
`M:draw(x, y, [opt])`	le righe da `x, y : w` (larghezza: i valori a destra), `c`, `sel_c`, `bar`, `dim`, `scale`, `gap`

Funzione	Descrizione
`lib.pause([opt])`	il **menu di pausa** che ogni gioco aveva: Start (Esc) lo apre quando `opt.when()` è vera (sempre se manca); RESUME, VOLUME (quello della console: sinistra e destra), le righe di `opt.rows`, QUIT (`opt.quit()`, se dato). `opt.color`, `opt.title`, `opt.scale`
`P:update()` / `P:draw()`	in `_update`: `if pause:update() then return end` (finché è aperto il gioco sta fermo); in `_draw`, dopo il gioco

```
local pause = lib.pause({ when = function() return S:is("play") end,
                          quit = function() S:go("title") end })
function _update()
  if pause:update() then return end
  lib.update(); S:update()
end
function _draw() S:draw(); pause:draw() end
```

bmlib: giocatori su una console

`btn(i, p)`, `stick(p)` e `controller(p)` leggono il giocatore `p`; bmlib aggiunge la schermata in cui si entra, lo schermo diviso e i colori. Il template *Versus 2D* dell'SDK è un esempio completo.

Funzione	Descrizione
`lib.PLAYER_COLORS`	i colori dei giocatori, quelli delle luci dei pad: 1 blu, 2 rosso, 3 verde, 4 rosa (come `controller(p).color`)
`lib.pads()`	i numeri dei giocatori che hanno ora un controller, in ordine (`{1, 3}`)
`lib.party([opt])`	la schermata in cui entrano i giocatori di una console: ognuno preme ok sul suo controller per esserci, indietro per uscire; Start di un giocatore che c'è fa iniziare, con almeno `min` giocatori (1); `max` (4); `join(p)`, `leave(p)`: funzioni chiamate quando uno arriva o se ne va. `P:update()` in `_update` restituisce i giocatori dentro (i loro numeri, nell'ordine di arrivo) quando il gioco inizia, altrimenti `nil`; `P:draw([x, y, w, h])` una scheda per ogni posto, nel colore del giocatore, con il suo controller e il tasto per entrare; `P.list` chi c'è ora
`lib.split(n, [opt])`	lo schermo tagliato in viste `{x, y, w, h}` per `n` giocatori (1-4): due affiancate (`vertical`: una sopra l'altra), tre o quattro negli angoli (con tre il basso a destra resta libero: una mappa, i punteggi); `gap` pixel tra loro (2); `x, y, w, h` la parte di schermo (tutto se mancano)

```
local party = lib.party({ min = 2 })
local views, cams

function _update()
  if not views then
    local who = party:update()              -- es. {1, 2}
    if who then
      views, cams = lib.split(#who), {}
      for i, v in ipairs(views) do cams[i] = lib.camera({ w = v.w, h = v.h }) end
    end
    return
  end
  -- ... cams[i]:follow(hero[i].x, hero[i].y)
end

function _draw()
  cls(0)
  if not views then party:draw(16, 80, SCREEN_W - 32, 200) return end
  for i, v in ipairs(views) do
    cams[i]:apply(v)
    cams[i]:map()
    draw_world()
  end
  clip()
  camera()
end
```

bmlib: suono, salvataggi, animazioni, colori

Funzione	Descrizione
`lib.jingle(notes, [voice, wave, vol])`	un breve motivo sulla voce (3): `notes = { {note, secs, [wave, vol]}, ... }`, la nota in Hz o per nome (`"C5"`), 0 o `"-"` una pausa; `lib.jingle(nil, voice)` lo ferma, `lib.jingling([voice])` dice se suona ancora
`lib.store([k, [v]])`	un campo del salvataggio della cartuccia (`save` / `saved`): `lib.store(k)` lo legge, `lib.store(k, v)` lo scrive (sulla scheda SD solo se è cambiato: non a ogni frame), `lib.store()` l'intera tabella
`lib.best(k, v)`	record `k` : `v` se lo batte (e lo salva), e `true` se è nuovo
`lib.frame(frames, fps, [t])`	l'elemento di `frames` che il tempo `t` (`time()` se manca) mostra a `fps` al secondo, in ciclo
`lib.anim(frames, fps, [loop])`	un'animazione tutta sua: `a:update()` avanza di un frame e restituisce l'elemento; `a.done` quando una che non si ripete (`false`) è finita; `a:reset()`
`lib.mix(c1, c2, t)` / `lib.shade(c, k)`	tra due colori `0xRRGGBB` / un colore `k` volte più luminoso (0 nero)

bmlib: il costruttore 3D

`lib.builder()` : una mesh fatta di pezzi convessi; ogni faccia di un pezzo si volge da sola verso l'esterno, quindi l'ordine degli angoli non conta mai.

Funzione	Descrizione
`B:piece(points, triangles, [colour])`	un pezzo convesso: `points = { {x, y, z}, ... }` , `triangles = { {i, j, k, [colour]}, ... }`
`B:box(x0, y0, z0, x1, y1, z1, colour, [top])`	una scatola tra due angoli; `top` : il colore della faccia superiore
`B:tetra(p1, p2, p3, p4, colour)` / `B:quad(p1, p2, p3, p4, colour, nx, ny, nz)`	un tetraedro / un quadrilatero piatto visto dal lato `nx, ny, nz`
`B:build()`	la mesh (`mesh()`); i metodi restituiscono `B` , quindi si concatenano

```
local house = lib.builder()
  :box(-1, 0, -1, 1, 1.5, 1, 0xC0A080)
  :piece({ { -1.1, 1.5, -1.1 }, { 1.1, 1.5, -1.1 }, { 1.1, 1.5, 1.1 }, { -1.1, 1.5, 1.1 }, { 0, 2.4, 0 }
         { { 1, 2, 5 }, { 2, 3, 5 }, { 3, 4, 5 }, { 4, 1, 5 }, { 1, 2, 3 }, { 1, 3, 4 } }, 0xA03020)
  :build()
```

Test dell'intera libreria: `make test-gameapi` (bmhost, `tests/gameapi/cart.lua` : ogni funzione con i suoi casi, anche i tasti con uno script) e `test_game_api` in QEMU.

bmnet: aprire la rete e la lobby

Le console si trovano nella **LAN** (broadcast UDP) o su **internet** attraverso un relay (`tools/overbit_relay.py`: una stanza passa ogni pacchetto alle altre console, come farebbe un broadcast). Una console **ospita** la partita e le altre **entrano**.

Funzione	Descrizione
`net.open([opt])`	apre la rete: `game` (4 caratteri, il gioco e la sua versione: i pacchetti degli altri giochi non si vedono), `port` (47320, uguale su ogni console), `relay` (`"name"` o `"name:port"`, 47310: su internet; senza, la LAN), `room` (4 caratteri, `"PLAY"`: le console di una stanza si vedono sul relay), `name` (il nome del giocatore). `true`, oppure `false` e il perché (nessuna rete)
`net.close()`	se ne va: gli altri lo sanno (in una partita l'input del posto diventa `false`; quando se ne va l'host la partita finisce per tutti), il socket si chiude. Chiamala anche da `_leave()`
`net.host([opt])`	questa console ospita una partita: `max` giocatori (2-8, 4), `info` (una riga che gli altri vedono nell'elenco). È il posto 1
`net.hosts()`	le partite trovate negli ultimi 3 secondi: `{id, name, players, max, info}`, una lista, ordinata per id
`net.join(id)`	chiede un posto alla console `id`: l'evento `"joined"` quando dice sì, `"refused"` se è piena o già iniziata
`net.peers()`	le console nella stanza o nella partita: `{id, name, seat, me}`, per posto

[DA COMPLETARE: come avviare il relay per una partita su internet; le fonti nominano solo `tools/overbit_relay.py --port N`]

bmnet: messaggi ed eventi

Funzione	Descrizione
`net.update()`	una volta in ogni `_update` : i pacchetti arrivati, gli annunci della lobby, i messaggi sicuri da rimandare. Restituisce gli **eventi** del frame, una lista di `{type = ...}` : `"join"` `( id, name, seat )`, `"leave"` `( seat, id )`, `"joined"` `( seat )`, `"refused"` , `"start"` `( seed, seat, seats, host, data )`, `"msg"` `( from, data, sure )`, `"lost"` `( why` : l'host se n'è andato), `"desync"` `( frame )`, `"error"` `( text )`
`net.send(data, [to])`	un messaggio (una stringa fino a 900 byte) a tutti o alla console `to` (un id): può andare perso o arrivare dopo uno più recente (posizioni)
`net.post(data, [to])`	un messaggio **sicuro**: ripetuto finché arriva, e ogni console riceve quelli di un'altra nell'ordine in cui sono partiti (chat, un turno, una scelta)

Usa `net.send` per ciò che viene sostituito dal messaggio dopo (posizioni) e `net.post` per ciò che deve arrivare (chat, un turno, una scelta).

bmnet: la partita in lockstep

Per i giochi d'azione, ogni console esegue tutto il gioco con lo stesso seed e viaggiano solo gli input dei giocatori. Un frame gira quando ci sono gli input di tutti per quel frame: l'host li raccoglie e li manda a tutti.

Funzione	Descrizione
`net.start([opt])`	(l'host) avvia la partita con chi c'è: `seed` (gli stessi numeri casuali su ogni console; uno a caso se manca), `delay` (frame tra una pressione e il suo frame: 4 nella LAN, 7 attraverso il relay), `data` (una stringa per tutti: le opzioni scelte). Tutti ricevono l'evento `"start"` (anche l'host, al prossimo `net.update()`)
`net.input(v)`	(lockstep) il mio input per il frame `net.frame + delay`: un intero a 32 bit (`net.pad(1)`). Una volta in ogni `_update` della partita, prima di `net.frames()`
`net.frames()`	i frame della partita che possono girare ora: `for f, inputs in net.frames() do ... end`, `inputs[seat]` l'input di quel posto (`false` quando il suo giocatore se n'è andato). Uno, due quando la console è indietro; nessuno finché manca un input (`net.stall`: secondi dall'ultimo frame girato)
`net.check(hash)`	un numero che riassume il gioco dopo il frame appena girato (posizioni, punteggi): le console lo confrontano e una differenza è l'evento `"desync"`. Basta ogni secondo
`net.pad([p])` / `net.unpad(v)`	i 16 tasti di `pad(p)` e la leva sinistra (8 bit per verso) in 32 bit / di nuovo tasti, `x`, `y` (-1..1)
`net.held(v, button)`	un tasto (`"a"`, `"left"` ... come `pad()`) premuto in un input di `net.pad`

ATTENZIONE

Le regole del lockstep. Il gioco della partita dipende **solo** dagli input dei frame e dal seed: numeri casuali con `lib.rng(seed)` (mai `math.random`, `time()` o `stat()`), niente della console (la camera, la qualità) dentro la simulazione, nessuno stato di gioco cambiato in `_draw`. Il disegno può essere diverso su ogni console.

- Due giocatori sulla stessa console in una partita in rete mettono i loro input nello stesso numero (16 bit ciascuno).
- `online(true)` viene attivato da `bmnet` quando la partita inizia: PS chiede conferma prima di uscire e chiama `_leave()`.

bmnet: campi ed esempio completo

Campi

Campo	Significato
`net.state`	`"off"`, `"lobby"`, `"hosting"`, `"joining"`, `"joined"`, `"playing"` o `"lost"`
`net.me`	il mio id
`net.seat`	il mio posto
`net.seats`	i posti della partita
`net.is_host`	`true` sull'host
`net.frame`	il prossimo frame
`net.seed`	il seed della partita
`net.delay`	il ritardo della partita in frame
`net.data`	la stringa data a `net.start`
`net.error`	il testo dell'errore per l'evento `"error"` (relay sconosciuto); anche "la partita è piena o già iniziata", "l'host se n'è andato" e "l'host non c'è più" (arrivano con gli eventi `"refused"` e `"lost"`). `net.open` lo riporta a `nil`
`net.stall`	secondi dall'ultimo frame girato, mentre `net.frames()` aspetta (azzerato in `net.frames()` e all'inizio della partita; cresce di 1/60 a ogni `net.update()`)

Un esempio completo

```
local net = require "bmnet"
local lib = require "bmlib"
local rng

function _init() net.open({ game = "MYG1" }) end
function _leave() net.close() end

function _update()
  for _, e in ipairs(net.update()) do
    if e.type == "start" then rng = lib.rng(e.seed) end
  end
  if net.state == "lobby" and btnp("a") then net.host({ max = 2 }) end
  if net.state == "hosting" and btnp("start") then net.start() end
  if net.state == "playing" then
    net.input(net.pad(1))
    for _, inputs in net.frames() do step(inputs) end   -- il gioco, uguale ovunque
  end
end
```

Provarlo sul PC

Due `bmhost` sullo stesso PC sono due console: `BMHOST_NET_ID=0` e `BMHOST_NET_ID=1`, con `--realtime`; il relay si avvia con `tools/overbit_relay.py --port N`.

`make test-bmnet` lo fa: la lobby, messaggi sicuri con un quinto dei pacchetti perso (`loss = 0.2` in `net.open`, solo per i test), una partita lockstep uguale su entrambe le console, un giocatore che se ne va; nella LAN e attraverso il relay.

Punti aperti

- **Tutorial: un passo di platform con bmlib**: come dipingere le tessere e impostare i loro flag negli editor; questo volume descrive solo la libreria
- **Tutorial: un passo di platform con bmlib**: immagine da fare: shots/api-lib-platformer.png
- **Tutorial: un gioco a due in rete con bmnet**: immagine da fare: shots/api-lib-lobby.png
- **bmnet: aprire la rete e la lobby**: come avviare il relay per una partita su internet; le fonti nominano solo `tools/overbit_relay.py --port N`